

FSD-Project

Ben Krüger, Erik Wiese, Anna Kugelmann, Martin Wellendorf

May 2025

Contents

1	Introduction	2
1.1	The Tasks	2
1.2	The Data	2
2	Task 1 - Parameter Tests and Comparison of the models	2
2.1	Hyperparameter Optimizer	2
2.2	Fully Connected Network	3
2.3	Convolutional Neural Network	4
2.4	Long Short-Term Memory Network	6
2.5	Comparison	8
3	Task 2 - Bias in the Dataset	10
3.1	Analysis with Respect to Metadata	10
3.2	Data Preprocessing	12
3.3	Data Augmentation	13
3.4	Debiasing Autoencoder	15
4	Summary	18
5	References	19

1 Introduction

1.1 The Tasks

Subject to this project is the classification of brakes in terms of functionality. We want to approach the project in a technical manner by using neural networks to determine whether a brake in a car is performing well or not. In case of defective brakes, it is also required to determine the configuration of faulty brakes among eleven in the dataset occurring possible configurations of defective brakes. For this classification task we want to use different types of neural networks, i.e., Fully Connected Networks (FCNs), Convolutional Neural Networks (CNNs) and Long Short-Term Memory Networks (LSTMs) in exercise A1.1. To avoid overfitting in task A1.2, we use the Random Split Validation method for training and validation and by adjusting certain hyperparameters, we might achieve better accuracies. In conclusion, we evaluate different combinations to compare the results in the end.

After implementing the described networks with certain hyperparameters we limit ourselves to the best performing model of each architecture and work out weaknesses, strengths and ideas for improvement. Therefore we investigate internal structures in the dataset such as possible correlations between data and metadata which might affect the overall performance of the networks drastically. Afterwards we take a brief look at the training data itself and how we can manipulate its range of values as well as its distribution. Such methods are called data preprocessing and data augmentation. Again, we will compare the accuracies of the models with and without the manipulation of the data. In A2.2 we are interested in a Debiasing Variational Autoencoder, which we are going to implement, train, evaluate its results and analyse the advantages and disadvantages in comparison to the other models. (Erik)

1.2 The Data

Additionally to the task it is important to get familiar with the data. The test data is structured in 6 columns and 1001 rows, whilst the train data contains 4053 rows. The first column is the sensor data, where each sample is a 2D-NumPy array of the shape (128,6). There are 6 columns of rotational velocity and acceleration along each axis, i.e.

- ax, ay, az for the acceleration along the x,y and z-axis in $\frac{m}{s^2}$
- rx, ry, rz for the rotational velocity around the x, y and z-axis in $\frac{^\circ}{s}$.

Each of these six values were collected for 128 distinct points in time. The second column of the data is the label, the actual condition of the brake. In the train data we can distinguish between 12 possible classes, from "all brakes work properly" which is denoted by 0-0-0-0 (class 0), to "all brakes have irregularities" as 1-1-1-1 (class 11). Next to the pure cases there are also all combinations of one defective brakes and three normal working brakes, i.e. 1-0-0-0 (class 4), 0-1-0-0 (class 5), 0-0-1-0 (class 1) and 0-0-0-1 (class 2), as well as all of two faulty and two intact brakes, so 1-1-0-0 (class 6), 1-0-1-0 (class 7), 1-0-0-1 (class 9), 0-1-1-0 (class 10), 0-1-0-1 (class 8), 0-0-1-1 (class 3), where 0 describes an intact and 1 an irregular brake. We can notice that the labels are fairly imbalanced. In the train data the majority of samples belongs to the 0-0-0-0 label with 1798 cases. Most of the labels describing one or two faulty brakes occur in contrast to the class 0 just between 200 and 300 times. The least amount of samples are from class 11, all brakes have irregularities, with 20 cases and class 9 and class 10 with 94 and 89 samples. Even stronger is the imbalance in the test data. One can only find five classes, with more than the half of the whole test set, 593 cases belong to the class 0. Beyond that we have to acknowledge that the other classes in the test data are the labels class 1, class 2, class 4 and class 5, so all cases with one defective brake, with around 100 appearances.

The third column specifies the exact model of the examined vehicle. The three further columns in the dataset characterizes the velocity in $\frac{km}{h}$, the mass in kg and the average of the deceleration in $\frac{m}{s^2}$, all measured during the process of collecting the data. (Erik)

2 Task 1 - Parameter Tests and Comparison of the models

2.1 Hyperparameter Optimizer

Countless hyperparameters may be varied in an FCN, CNN, or LSTM architecture. The comet library provides an optimizer class, which can search the space of possible hyperparameter combinations and find

an optimal combination with respect to some specified metric. The following table shows the architecture-independent hyperparameters we searched.

hyperparameter	type	range	explanation
batch size	integer	$\{16, 32, 64, 128, 256\}$	Number of data samples that are processed simultaneously within one training step.
learning rate	float	$[10^{-5}, 10^{-2}]$	Learning rate of the optimizer used in training.
dropout	float	$[0, 0.5]$	Proportion of neurons that will be randomly selected and whose output will be ignored (i.e., set to zero).
optimizer	categorical	$\{\text{adadelata, adam, adamw, lion, rmsprop, sgd}\}$	Optimizer to be used in training.

Table 1: Space of architecture-independent hyperparameters

The hyperparameter optimizer searches the space of these hyperparameters as well as architecture-specific hyperparameters (cf. Sections 2.2 to 2.4) with the help of the Bayes algorithm (cf. [3]). Its objective is to find a hyperparameter combination that minimizes the *average validation loss* according to the *random split validation method*.

More precisely, the training dataset is split randomly into a training section and a validation section (in our case with a proportion of 80% training and 20% validation). A new model is instantiated, trained on the training section, and evaluated on the validation section, yielding a (total) validation loss. This process is repeated k times, where $k \in \mathbb{N}_+$. The *average validation loss* is the average of the validation losses over all k splits. We decided to use $k = 5$ as a balance between robustness and computational cost.

To lower the computational cost even more we also implement Early Stopping for all models, with respect to the validation loss and a patience of five, meaning that if the loss does not improve during five epochs, the training is stopped and the weights of the best performing model are restored. Additionally we set a maximum time limit of five minutes per data set. (Martin)

2.2 Fully Connected Network

Fully Connected Networks (FCN) consist of multiple dense layers, connecting every neuron of one layer with every neuron in the subsequent layer. We chose a maximum of six layers to reduce computational time and ensure comparability between the three model types. Additionally to the number of dense layers, we added the following parameters describing each layer as hyperparameters to the optimizer.

hyperparameter	type	range	explanation
number of Dense layers	integer	$[1, 6] \cap \mathbb{Z}$	Number of Dense layers used in the model.
units	integer	$\{32, 64, 128, 256\}$	Number of output features of the Dense layer.
activation	integer	$\{\text{tanh, sigmoid, relu}\}$	Activation function for the output.

Table 2: FCN architecture and space of hyperparameters per layer

The optimizer chose 20 different settings of hyperparameters, with an average training duration of 18.59 seconds per data set. Although the time spans varied between 4.0 and 44.8 seconds, no strong correlation between the training time and the models performance, i.e. its average accuracy and loss, shows. The trained models perform quite well on the validation set with six of them reaching over 98% and an average of 81.922% accuracy. On the contrary, the best average classification of the test data is only about 60% accurate.

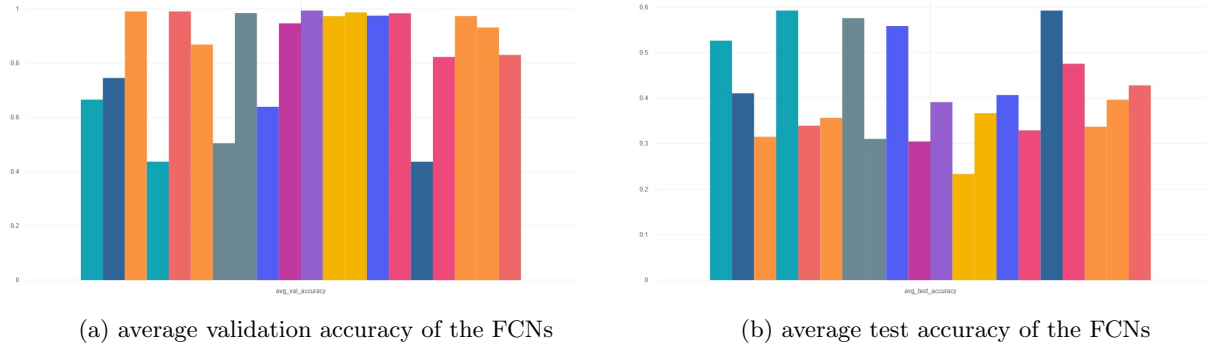


Figure 1: FCN performance

For choosing the "best" hyperparameter setting, we first have to define what determines the performance of the model. Keeping it as close to real life applications as possible, we decided to only consider the accuracy and loss on the validation data, since the evaluations on the test set would be unknown. Using this metric the best performing FCNs are listed below with their corresponding choice of hyperparameters. We could find almost no similarities in the parameter settings, the only correspondence seems to be a higher numbers of units per dense layer.

Hyperparameters	Best model	Second best model	Third best model
Validation accuracy	99.41%	99.06%	99.04%
Validation loss	0.01682	0.04012	0.04484
Test accuracy	39.12%	33.93%	31.51%
Test loss	4.876	3.158	9.18
Duration	36s	33s	101s
Number of dense layers	4	1	2
Units per layer	64, 64, 256, 128	64	64, 32
Activation per layer	sigmoid, relu, tanh, tanh	tanh	sigmoid, relu
Optimizer	rmsprop	adamw	rmsprop
Learning rate	0.00204423	0.00521861	0.00191442
Dropout	0.0423074	0.300586	0.360588
Batch size	64	128	8

Table 3: Three best models with FCN architecture and their parameters

As seen in table 3 the "best" models perform much worse on the test set with accuracies only around 30 – 40%. At the same time the loss is comparatively high, which gives a hint, that we are not stuck in an optimum but can increase the performance a lot. Furthermore none of the models reach more than 60% on the test data. This fact and the discrepancy between the performance of the trained models on the validation and test data is further explored in section 2.5. (Anna)

2.3 Convolutional Neural Network

Our Convolutional Neural Networks (CNN) consist of multiple convolutional layers, each followed by a pooling layer, and several dense layers. Therefore we added the number of convolutional and dense layers as hyperparameters to our optimizer.

hyperparameter	type	range	explanation
number of Conv layers	integer	$[1, 3] \cap \mathbb{Z}$	Number of convolutional layers used in the model.
number of Dense layers	integer	$[1, 3] \cap \mathbb{Z}$	Number of Dense layers used in the model.

Table 4: CNN architecture - Possible numbers of layers

For each convolutional layer the following hyperparameters can be chosen.

hyperparameter	type	range	explanation
filters	integer	{16, 32, 64, 128}	Number of filters of the convolutional layer.
conv activation	categorical	{tanh, sigmoid, relu}	Activation function.
kernel size	categorical	{(3,1), (3,2), (3,3), (5,5)}	Size of the kernel used in the convolutional layer.
pooling	categorical	{avg, max}	Type of pooling used in the model.

Table 5: Space of hyperparameters per convolutional layer

By using not only quadratic but also unsymmetrical kernels we hoped to extract features in one timeline of measurements more easily. Nevertheless the limited number of timesteps and measurements led us to smaller kernel sizes with the maximum being 5×5 . Because of the dimensional difference in the input data we chose to only pool the data in one direction, i.e. a pool size of 2×1 . Therefore we only pool along the time series and not along the different measurements.

Analogously to the convolutional layers, each dense layer can be described by the following hyperparameters.

hyperparameter	type	range	explanation
units	integer	{32, 64, 128, 256}	Number of output features of the Dense layer.
activation	categorical	{tanh, sigmoid, relu}	Activation function for the output.

Table 6: Space of hyperparameters per Dense layer

The 20 models with different hyperparameter settings chosen by the optimizer, trained for 140.96 seconds per data set on average. The longer duration does not imply a better performance on the validation data set, but shorter trained models (about 25 seconds per dataset) almost never reach high accuracies on the validation data. As seen in the figure 2 six models reached validation accuracies over 98%, which is similar to the FCN performance. When looking at the validation and test accuracy one immediately notices two thresholds. While most of the models score the same accuracy of 43.67% on the validation data, the same models reach exactly 59.24% on the test data set. This is not only unexpectedly high, but also the same value the best CNN models reach. Those thresholds are explained in section 2.5.

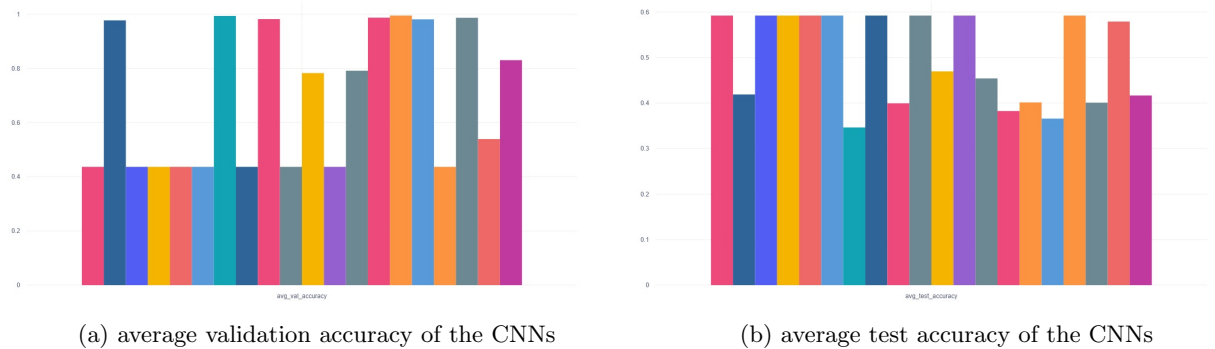


Figure 2: CNN performance

With the same definition we used before, we identify the three best models with CNN architecture. They all stick to lower batch sizes, but besides that they do not share hyperparameters. While the first and the third model used the SGD optimizer, the second one was optimized using rmsprop, which seems to be a lot faster than SGD, based on these three models.

Hyperparameters	Best model	Second best model	Third best model
Validation accuracy	99.51%	99.33%	98.74%
Validation loss	0.01826	0.02565	0.07012
Test accuracy	40.14%	34.64%	38.26%
Test loss	4.287	5.632	2.711
Duration	951s	277s	1511s
Number of dense layers	2	2	1
Units per layer	32, 256	32, 128	128
Activation per layer	sigmoid, tanh	relu, relu	tanh
Number of conv layers	2	3	2
Number of filters	64, 64	64, 32, 128	128, 16
Kernel sizes	(3,3), (3,2)	(5,5), (3,1), (3,1)	(5,5), (3,3)
Activation per layer	sigmoid, tanh	tanh, relu, relu	sigmoid, relu
Optimizer	SGD	rmsprop	SGD
Learning rate	0.00995662	0.00159913	0.00130245
Dropout	0.185555	0.16426	0.353287
Batch size	8	16	32

Table 7: Three best models with CNN architecture and their parameters

(Anna)

2.4 Long Short-Term Memory Network

Our LSTM models always consist of several LSTM layers followed by several Dense layers. Thus, we add the following hyperparameters to Table 1.

hyperparameter	type	range	explanation
number of LSTM layers	integer	$[1, 4] \cap \mathbb{Z}$	Number of LSTM layers used in the model.
number of Dense layers	integer	$[1, 3] \cap \mathbb{Z}$	Number of Dense layers used in the model.

Table 8: LSTM architecture - Possible numbers of layers

For each LSTM layer, we additionally include the following hyperparameters.

hyperparameter	type	range	explanation
units	integer	$[1, 64] \cap \mathbb{Z}$	Number of output features of the LSTM layer.
activation	categorical	{relu, tanh, sigmoid}	Activation function.
recurrent activation	categorical	{relu, tanh, sigmoid}	Activation function for the recurrent step.
go backwards	categorical	{true, false}	Whether the sequence should be processed forwards or backwards.

Table 9: Space of hyperparameters per LSTM layer

Finally, as we did before, we include the following hyperparameters for each Dense layer.

hyperparameter	type	range	explanation
units	integer	{32, 64, 128, 256}	Number of output features of the Dense layer.
activation	categorical	{tanh, sigmoid, relu}	Activation function for the output.

Table 10: Space of hyperparameters per Dense layer

With hyperparameter ranges similar to the above one notices that the experiment losses often degenerate to NaN. Upon closer investigation one can find a correlation between the number of occurrences of the relu function as activation or recurrent activation in lstm layers and this invalid loss.

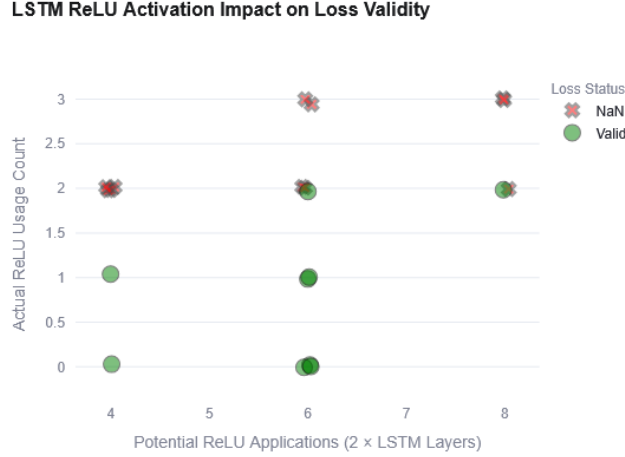


Figure 3: Correlation between occurrences of relu as activation or recurrent activation in LSTM layer and degenerate loss.

This behavior can have multiple possible explanations, for example:

1. Since relu is unbounded, the long length of the gradient flow (induced by the length of the time series) can cause gradients to "explode" and reach NaN (cf. [1]).
2. Negative inputs into relu yield constant 0, possibly resulting in information loss (dying relu, cf. [7]).

The second issue may be mitigated by using

- leaky relu

$$f(x) = \begin{cases} \alpha x, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

where $\alpha \in (0, 1)$,

- or exponential linear unit (elu)

$$f(x) = \begin{cases} \alpha(e^x - 1), & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

where $\alpha > 0$ [2].

However, using leaky relu or elu instead of relu did not mitigate the occurrence of NaN as loss, indicating that dying relu was not the cause of this issue.

Consequently, in the following hyperparameter optimization we will only consider tanh and sigmoid. The modified search space per LSTM layer looks as follows.

hyperparameter	type	range	explanation
units	integer	$[1, 64] \cap \mathbb{Z}$	Number of output features of the LSTM layer.
activation	categorical	{tanh, sigmoid}	Activation function.
recurrent activation	categorical	{tanh, sigmoid}	Activation function for the recurrent step.
go backwards	categorical	{true, false}	Whether the sequence should be processed forwards or backwards.

Table 11: Modified space of hyperparameters per LSTM layer

The hyperparameter optimization process chose 38 different hyperparameter combinations. The training times varied significantly between these architectures: the combination with the longest training time (average over the 5 random splits) took 391.6 seconds, while the fastest architecture took 9.2 seconds. On average,

training one model on one split took 108.9 seconds.

The following graph shows the *average validation accuracy* as well as the *average test accuracy* of the 38 models.

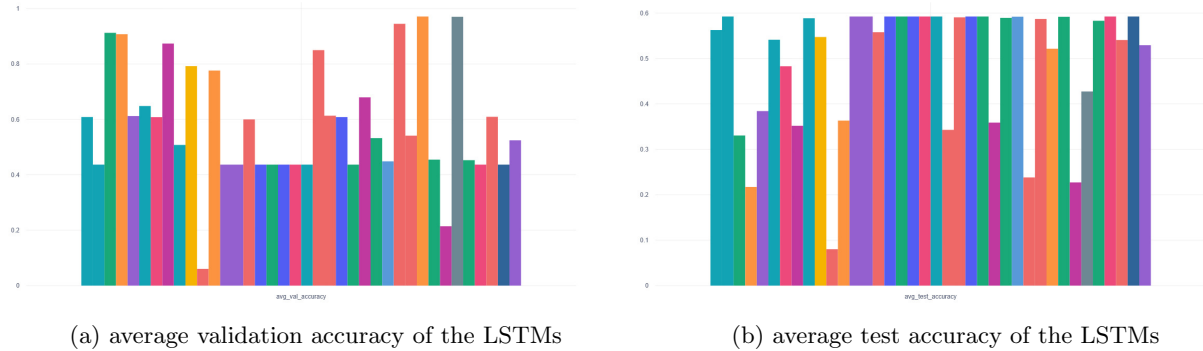


Figure 4: LSTM performance

Once again we notice the same thresholds as seen in Section 2.3; 19 out of 38 models score the same accuracy of 43.67% on the validation data and 59.24% on the test data set. See Section 2.5 for an explanation. Thus, it makes no sense to consider "the best" model concerning the average test accuracy. However, concerning the average validation accuracy, the following table summarizes the hyperparameter configurations of the three best-performing models.

Hyperparameters	Best model	Second best model	Third best model
Validation accuracy	97.13%	97.04%	94.50%
Validation loss	0.1124	0.1406	0.2099
Test accuracy	52.14%	42.74%	23.82%
Test loss	3.0087	3.1761	3.3256
Duration	120s	105s	512s
Number of LSTM layers	1	2	2
Units per LSTM	42	36, 43	16, 23
Activation per layer	sigmoid	sigmoid, tanh	tanh, sigmoid
Recurrent activation per layer	tanh	tanh, tanh	tanh, tanh
go'backwards per layer	False	True, True	True, True
Number of Dense layers	2	2	2
Units per layer	64, 256	32, 256	32, 64
Activation per layer	tanh, sigmoid	relu, tanh	relu, relu
Optimizer	rmsprop	rmsprop	rmsprop
Learning rate	0.00900122	0.00562781	0.00411315
Dropout	0.0064	0.1704	0.3075
Recurrent dropout	0.4502	0.2468	0.2064
Batch size	32	128	8

Table 12: Three best models with LSTM architecture and their parameters

(Martin)

2.5 Comparison

When considering the average validation accuracy of the three different neural network architectures, one notices, that none of them reaches more than 60%. Moreover, a lot of the models seen in figures 1b, 2b, 4b have the exact same average test accuracy of 59.24%. Additionally those models reach precisely 43.67% on the validation data. To further explore this phenomenon we had a look at the confusion matrices of the corresponding models. One example is presented in figure 5 below.

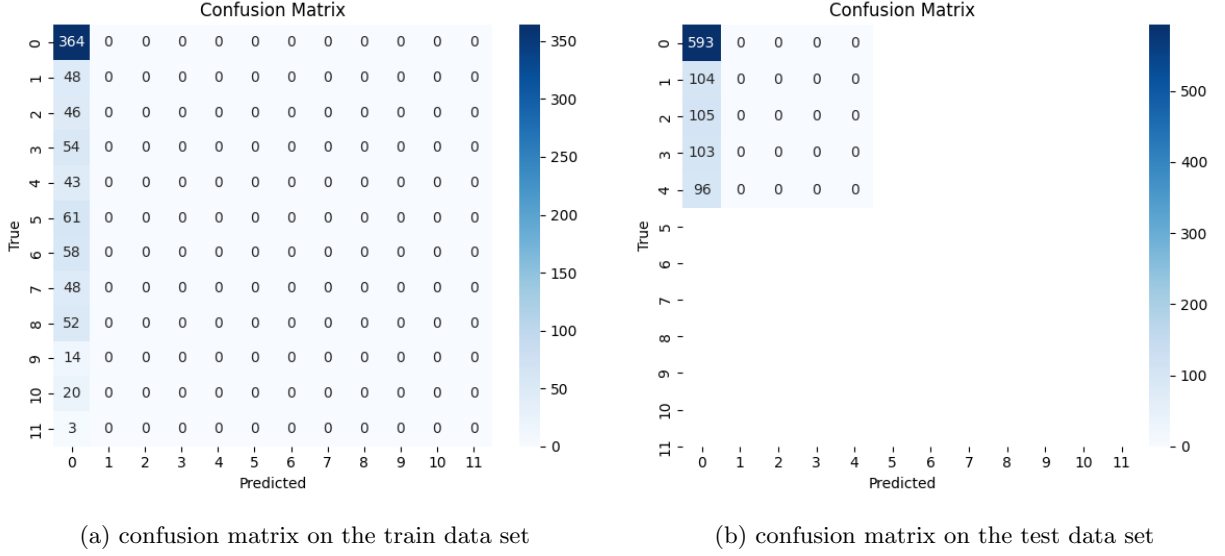


Figure 5: FCN performance

All the models reaching the observed threshold are guessing label zero every time. Therefore when classifying the test data they reach around 60% accuracy, the prevalence of this label in the test set. Note that label zero is less likely in the training data, with around 44%. This fact explains the duality we could see in some of the models and very prominently when looking at the CNNs in figure 2. Models that did not perform well on the validation data suddenly score much higher on the test data. The only reasons for this phenomenon are the different quantities of data labelled zero and the bias in the data, where label zero is by far the most common class. Therefore only guessing the first label reaches quite good accuracies.

Of course this is not the desired behaviour of our networks but more a result of the bias in our data and the distinct differences between the train and test data. Not only the methods of retrieving the data vary significantly between these sets, but also the car models, their weights and their initial velocity. This topic will be further explored in 3.1.

In the following graphic we have a closer look at the label distribution across the two data sets. In both sets of measurements label zero is dominating all the other labels by far, with 44% percent in the training data and 59% percent in the test data. Because of the strong bias the models start to learn patterns, in our case just guessing label zero, instead of learning the underlying features. Consequently the models are less generalizable and consistently makes errors on other labels. This behaviour is also rewarded, since label zero is correct most of the time. Furthermore the test set is representing a lot less labels then the training data.

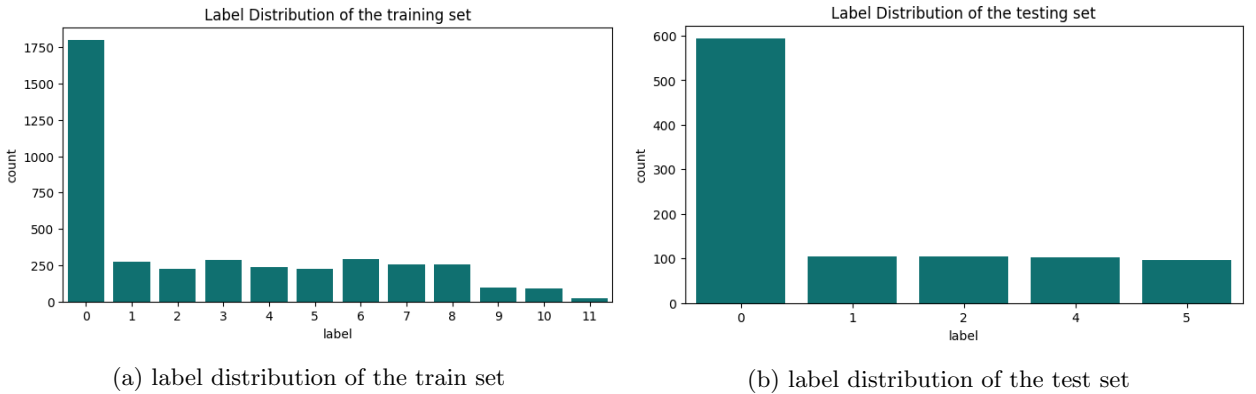


Figure 6: label distributions

The limited capability of all three models to capture the underlying features of the braking process makes it impossible for us to prefer one neural network over the others, leading us to choose not one best model.

On the contrary we decided on a best FCN, CNN and LSTM model using their accuracy on the validation data set. The models can be seen in the first columns of the tables 3, 7 and 12. Although the FCN and CNN reached both a higher accuracy on the validation set, the LSTM model performs around 12% better on the test set. One reason could be some sort of overfitting, where the FCN and CNN learn the training data too well and are not able to generalize it to the noticeably different test data. This effect might have been prevented for the LSTM by the early stopping we introduced to the training. Due to long computation times this boundary was mostly reached by the LSTM and not by the CNN and especially not by the FCN. When it comes to computational time the FCN are definitely the fastest, with only 36 seconds compared to 120 seconds of the LSTM and almost 1000 seconds for the chosen CNN. One has to keep in mind that the LSTM ran on a GPU, while the other two were computed on a CPU. (Anna)

3 Task 2 - Bias in the Dataset

3.1 Analysis with Respect to Metadata

To analyse the models further, we are splitting the validation and test data into different categories and evaluate each models performance on the subdata. We decided to part the data into the different masses, the deceleration averages, the initial velocities and the cars model, binning the values if needed. When comparing the performance in relation to different metadata it is important to keep in mind how the overall performance of the models are. The FCN and CNN reached approximately 40% accuracy, whereas the LSTM was able to classify around 52% of the test data correctly.

Considering the validation data all three models perform very well on all the categories, which is not surprising, since the data is closely related to the training data. There are only a few exceptions, that lie under 98%, such as the car model "Variant Comfortline", which is slightly less correctly classified, as well as the mass of 1530kg. The FCN and CNN only perform around 97%, while the LSTM only reaches 88%. This effect is easily explained, since the car weighing 1530kg is the "Variant Comfortline". Also for the average deceleration of 3 the models perform slightly worse. Note that these effects are stronger for the LSTM compared to the FCN and CNN structure, which could be another sign for less overfitting of the LSTM.

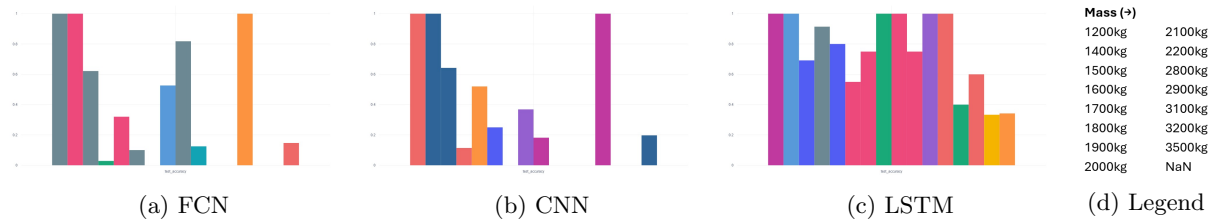


Figure 7: Performance of the models on test data split by mass

The split into metadata revealed some significant differences in the test and training data. While the training set contained only three different weights ranging from 1341kg to 1594kg, the test data set presented much more variety from 1200kg to 3500kg. This variety has to be considered with reservation, since about 38% of the measurements lie around 1500kg and another 46% have no weight assigned. This leaves only 16% of the data that contribute to the wide spread range of weights. Because of the similarity to the training data we assume that the models perform well on weights around 1500kg. This is mostly the case with above average accuracies of 60-70% of the three models. On the opposite the accuracy for the other weight classes seem rather arbitrary. Since the number of measurements per class are under 35 and mostly even under 10, the accuracies classifying these measurements vary a lot between the three models. On this occasion the FCN and CNN score very similar, while the LSTM shows mostly different behaviour. At last we have to focus on the large group of data having no assigned weight. Naturally we can not predict any outcome on those, but as the largest group of measurements they score always under 35%, with 15% on FCN, 20% on CNN and 34% on the LSTM.

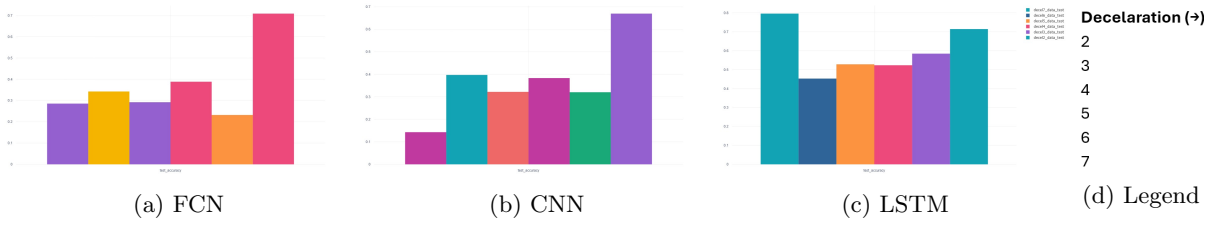


Figure 8: Performance of the models on test data split by the deceleration average

The deceleration averages occurring in the train and the test set are basically identical, with a range from $(-3) - (-7) \frac{m}{s^2}$. While the occurrence in the training set is centred around 5, the test set has an almost equal distribution between those values, excluding the seven measurements binned in the class $(-2) \frac{m}{s^2}$. Therefore the number of measurements in the train set, having a deceleration average of seven is quite limited to only 15. Surprisingly this does not have a effect on the performance on the test set, as all three models reach more than 65%, which lies above their overall performance. The other deceleration classes are classified around average, with the LSTM always outperforming the FCN and CNN. All in all the metadata about the deceleration average does not seem to give a hint what data is harder to classify and which is not.

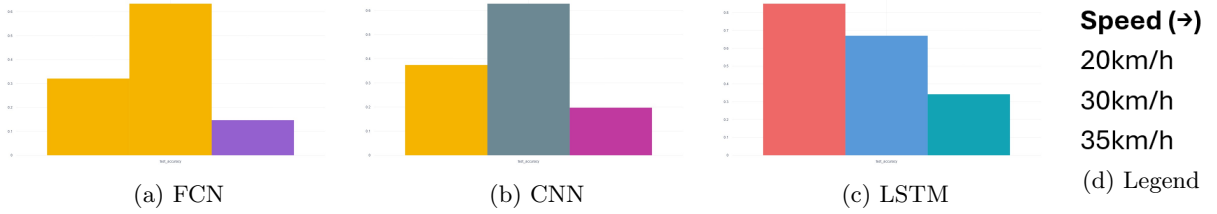


Figure 9: Performance of the models on test data split by initial speed

On the one hand the initial velocity in the training data is limited to a maximum of $24 \frac{km}{h}$, with most of the measurements (58%) starting at $20 \frac{km}{h}$ and another 16% at $19 \frac{km}{h}$ as well as 20% at $21 \frac{km}{h}$. Therefore the vast majority of measurements is centered around those values. On the other hand the test data has only velocities of 20,30 or $35 \frac{km}{h}$. For that reasons we expect the models to classify the lower speed data in the test set very well, while the classification of the others could be fairly difficult. These assumptions are only fulfilled by the LSTMs performance as seen in figure 9, while the FCN and CNN surprise with low accuracies at $20 \frac{km}{h}$ initial speed. They only reach about 30-40% accuracy, while the best classified class starts at $30 \frac{km}{h}$ and reaches over 60%. Nevertheless the class corresponding to an initial speed of $35 \frac{km}{h}$ is only classified correctly in 15-20% of the cases, as expected far under the average accuracy.

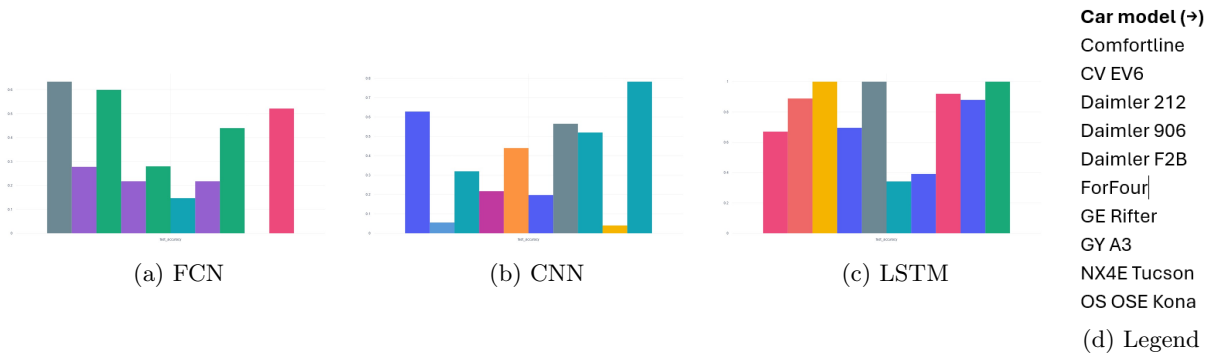


Figure 10: Performance of the models on data split by model of the car

The training data is separated in only three car models, while the test data contains ten different models. Nevertheless most of those contain only 25 different measurements, just two classes have a significant amount of measurements. Therefore we focus only on those two in the analysis. The "Variant Comfortline" is the

only model present in both data sets, that is why we expect the models to classify this class relatively well. Our expectation is met, as all three model architectures reach 60-70% on this class, a performance above average. The second major class in the test set, containing 46% percent of the data, is the "ForFour", where all the models perform under the average accuracy. The FCN barely reaches 15%, the CNN is with almost 20% slightly better and even the LSTM stays far under the expectation with only 34%.

The analyses made above present just a small portion of the investigations that could be made about the models. For example one could look at the label distribution per metadata set, which could give more insights about why certain groups are classified more easily than others. (code by Erik, analysis by Anna)

3.2 Data Preprocessing

To preprocess the data, we used two popular techniques: normalization and standardization. Both of them aim to rescale and redistribute the data to improve learning rates and help with debiasing. Both techniques reduce the effect of different magnitudes in multiple data streams. These different magnitudes might impact the convergence rate of the neural network, since features of higher magnitude disproportionately influence the gradient. Bias might also arise from this effect, as the network overvalues features which are connected to data with larger scaling. Normalization, specifically min-max-normalization, takes in a one dimensional set of data points and projects the minimum to zero and the maximum to one. All other points are projected in the way that the relative distance to the minimum and maximum is preserved. This is equivalent to the formula $\hat{X} = \frac{X - X_{min}}{X_{max} - X_{min}}$, where $\hat{X}$ is the newly obtained data point from X . Note that for division by zero we have that $X_{min} = X_{max}$, which is equivalent to all data points being equal. So by convention, we will set every point to zero in that case.

Standardization processes the dataset by centring the values around zero with a standard deviation of one. Each data point is adjusted based on the mean and standard deviation of the dataset, preserving the overall distribution shape while transforming its scale. This follows the formula $\hat{X} = \frac{X - \mu}{\sigma}$. Similar to before $\sigma = 0$ would lead to the division of zero by zero, since it means that any value is the same as the average. It also corresponds to the case where all data points are identical. Once again, we simply set every value to zero.

For our application we used various versions of these techniques, tailored to the structure of our data. We were not focusing on improvements for the convergence rate, as that has not been a remarkable issue in our first models. To try and find the best application for our models, we standardized and normalized every experiment dataset along the timeline, over all acceleration types and both at once. In other words the processing was applied along the rows, columns and the whole array. For the sake of exploration we also applied the technique to the whole dataset as a tensor. As expected, affine transformation along the whole set does not yield any of the benefits we try to obtain by rescaling, which is why it will not be part of further evaluation. This leaves us with two basic techniques, applied three different ways to three neural networks. From left to right the following diagrams show the average test accuracy on data of the type: unprocessed, array standardized, array normalized, row standardized, row normalized, column standardized, column normalized.

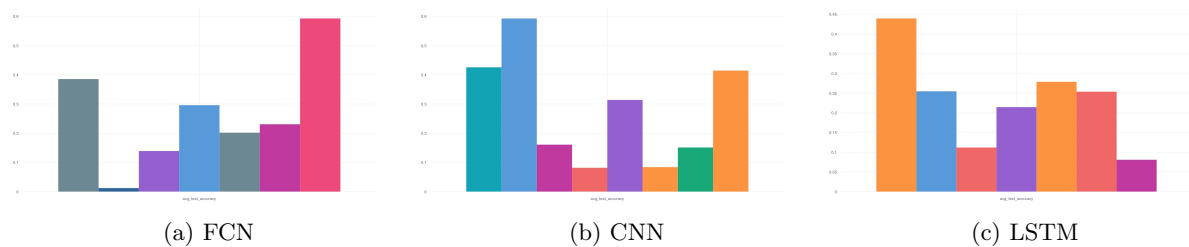


Figure 11: Average test acc of the models after preprocessing

Unfortunately across all eighteen cases there emerged two distinct outcomes. Whenever the model achieved higher accuracy compared to the networks without preprocessing, it was due to reaching the "optimal" strategy of guessing label zero as already described. This result is in complete contrast to the goal of minimizing bias, as this strategy is very specific to the over representation in our data and the least generalizing approach imaginable. On the other models we also did not observe an improvement in generalization, but actually a significant decrease with accuracy dropping from around 0.4 to less than 0.1. One possible explanation to the adverse effect is that the processing conceals properties in the data, that are actually important for the

recognition of defects. For example, the most important characteristic of one label might be a significant extremum along one time series. This extremum would be flattened, compared to the overall data in the attempt of debiasing. Consequently, the model is never able to train on this key feature. Additionally, there is no obvious pattern arising, as of which combination of processing technique and or data dimension processed is superior. The diagrams show that there is not one column that always stands out by being high/low at every network. What can be said, is that the outcome changes drastically based on that decision, which can be seen by the high variance in accuracy within one network. How much this outcome indicates a disadvantage of preprocessing or whether it is simply a symptom of other problems in the dataset could be subject to further research. (Ben)

3.3 Data Augmentation

The general idea behind data augmentation is to vary certain parts of the given data slightly such that the general structure is preserved. The underlying intention of performing data augmentation is on the one hand to get a more diverse set of data so the network might be more robust to slight deviations. On the other hand it is reasonable in the first place to have classes of roughly the same cardinality in the training process, thus the network is not biased in terms of appearances. In the context of the given task it is pointless to use data augmentation techniques for images. Instead it is much more convenient to use techniques of signal data augmentation also used for data augmentation for audio. Having the task in mind you can think of two general approaches for the data augmentation. Firstly it is quiet reasonable to consider such manipulation of the data which fits into the given physical and logical frame. So this new data might occur under different conditions but is not unrealistic at all. The other approach we want to try is to augment the data with no particular logical background in mind.

Therefore we start with the data augmentation by just adding a random gaussian noise term to each perceived point. In some sense it is reasonable in the setting since the possibility of mild deviations of the data are omnipresent due to differences of slight margin in the setup or just by the effect of disregarded physical influence. The second idea we used is the augmentation by scaling the amplitude of the series of data points. One can argue that this method as well as adding noise is quite practical since the internal process of the series is preserved. To experiment a bit with the data augmentation we also implemented two augmentation function which are not as straightforward as the prior two approaches.

Firstly the ZoomIn function generates a synthetic series out of the given one by taking half the points and determine the linear interpolation at the midpoints between two neighboring measurements. Therefore one can zoom into the series of points in the sense that only half of the time interval is viewed by the network. A further out of the air approach is to flip the series of points so you get a inverse array of the original one. Both attempts seem a bit far fetched from a human standpoint, but we analyze how they perform in comparison to the other ones. All augmentations were performed by iteratively augment the sensor data of the minor classes until the cardinality of all classes are equal and they have the same amount of samples as the major class 0. Therefore it is expectable to get exact duplicates of some samples (except by augmenting with Gaussian Noise). For instance we use the data of class 11 out of the train data at least 88 times for augmentation, since the original set with this label only contains of 20 samples.

We augment the train data in the explained manner for each the best FCN from 2.2, the best CNN from 2.3 and the best LSTM from 2.4. In theory the augmentation functions are implemented in a way, that again, there are various hyperparameters to choose from. For AmplitudeScaling we choose the factor 1.1 to scale the data with. Regarding the NoiseAdding function we decided to add a normal distributed term with mean 0 and standard deviation 1. For the ZoomIn function, we have agreed on a central zoom, which means that the selected hyperparameter is 32, which refers to the index of the array from which half of the data is extracted. For the augmentation function Flip no hyperparameter can be configured. We do not optimize the aforementioned hyperparameters, so there might be better results with other configurations possible. Firstly we take a look at the augmentations for the FCN, which performance is summarized in the following table.

FCN with augmentation	AmplitudeScaling	NoiseAdding	ZoomIn	Flip
Validation accuracy	99.94%	99.22%	99.85%	99.91%
Validation loss	0.00289	0.03199	0.00625	0.00457
Test accuracy	36.42%	34.41%	35.00%	38.84%
Test loss	6.532	6.564	5.745	6.11
Duration	100s	75s	81s	124s

Table 13: FCN with different data augmentation methods

On the validation data the FCN performs a 99.41% accuracy without any augmentation. The FCN with augmentation can match this performance, especially the augmentation with amplitude scaling and flipping with a validation accuracy of 99.94 % and 99.91% are remarkable. Also in terms of the validation loss manages the augmented FCN to get as good results as the best FCN. Just augmenting by noise adding has a higher loss than the FCN. After all is the performance on the validation data very satisfactory, but the test accuracy of the augmented networks can not match the test accuracy of the best FCN of 39.12%. Just the augmentation with Flip with an accuracy of 38.84% is the closest. Additionally to the worse accuracy, we can identify, that the test loss of all augmentation methods is higher than the loss of 4.876 of the default model.

We can also take a look at the performance of the augmented CNN.

CNN with augmentation	AmplitudeScaling	NoiseAdding	ZoomIn	Flip
Validation accuracy	99.96%	99.33%	99.72%	99.77%
Validation loss	0.00303	0.02097	0.01177	0.00981
Test accuracy	41.44%	40.30%	36.40%	36.18%
Test loss	4.148	4.399	3.571	3.648
Duration	1529s	1526s	1525s	1524s

Table 14: CNN with different data augmentation methods

We see, that the accuracy on the validation set is fairly high again. While the best CNN reaches an accuracy for the validation data of 99.51% the CNN with augmented data performs in three of four cases a better accuracy and in all cases an accuracy of over 99%. For the CNN we get also good results regarding the accuracy on the test set. Without data augmentation the best CNN is capable to perform a test accuracy of 40.14%, with data augmentation we get results above this accuracy with AmplitudeScaling (41.44%) and NoiseAdding (40.30%). ZoomIn and Flip fell off slightly by test accuracies around 36%.

Lastly, we analyse the results of the augmented LSTM.

LSTM with augmentation	AmplitudeScaling	NoiseAdding	ZoomIn	Flip
Validation accuracy	99.52%	97.29%	98.29%	98.93%
Validation loss	0.01596	0.09208	0.05106	0.03502
Test accuracy	45.73%	47.55%	53.37%	46.41%
Test loss	3.995	3.118	2.345	3.373
Duration	1522s	1416s	1524s	1524s

Table 15: LSTM with different data augmentation methods

Although the best LSTM without any augmentation performed just a validation accuracy of 97.13%, the LSTMs with augmented data are capable of getting higher results, in particular the LSTM with scaled data with an accuracy of 99.52%. The test accuracy of this models are also fairly solid, with performances from 45% to 48%. With augmentation by zooming the LSTM even achieves an accuracy on the test data of 53.37%. An interesting observation we can do, is that the validation loss of the augmentation NoiseAdding across all three models is among the other losses the highest by a great margin. This might be correlated to the randomness of this augmentation which leads to a different and more difficult task for the model to get the characteristics of the data. Nonetheless are the validation losses in contrast to the losses of the default models remarkably smaller.

We can also make a comment on the duration. All augmented models have very long durations compared

to their default versions. The augmented FCN needs in average around 100s, while the best FCN just needs 36s. The durations of the CNN and the LSTM with augmentations is always around 1500s and a lot higher than the standard CNN and LSTM. All these longer durations might correlate with the fact, that more data has to be viewed and analysed by the network.

In conclusion, we can see, that the augmentation is very useful in theory, in our case, this effect shows itself in parts. The effect might be greater, if all labels from the train data also appear in the test data. The validation accuracy is always quite high and some of the test accuracies are higher than the test accuracy of the default model. A downside might be the larger amount of data and a longer duration, which might be even more problematic if the original dataset is even bigger and more complex. Among the augmentation methods there is no clear superior method for us, since the best performing augmentation differs from network to network. Even the ZoomIn and Flip data augmentation methods perform reasonably good. (Erik)

3.4 Debiasing Autoencoder

In Section 3.3, we mentioned that data augmentation methods are used for minor classes of the training data such that all classes have the same cardinality and, thus, their class features are represented equally. However, this technique can only mitigate bias between known and preselected classes. In contrast, bias may occur within a class, possibly due to features that have not been identified. For this reason, we now pursue the approach of a debiasing variational autoencoder (DB-VAE), which was first proposed in [4] and introduced in our lecture in [5]. With the help of a variational autoencoder (VAE), features of the data samples are learned during training in the form of latent variables. The distribution density of each latent variable is then approximated via a histogram, and a new distribution is created by combining the inverses of the histogram densities of all latent variables (using multiplication as in [4] or maximization as in [5]). During training, batches are drawn randomly according to this new distribution, ensuring that data samples with underrepresented features are chosen more frequently.

Like a VAE, the DB-VAE consists of an encoder and a decoder part. The encoder part is a CNN with an architecture very similar to the best CNN from Section 2.3, but it has 12 additional outputs – the logits used for classification. The decoder part uses transposed convolution layers (also known as deconvolution layers) that mirror the structure of the convolution layers in the encoder.

The loss used for training is given by

$$\mathcal{L}_{\text{latent}}(x) = \frac{1}{2} \sum_{j=1}^d (\exp(\log \sigma_j^2) + \mu_j^2 - 1 - \log \sigma_j^2) \quad (1)$$

$$\mathcal{L}_{\text{reconstruction}}(x, \hat{x}) = \frac{1}{768} \sum_{i=1}^6 \sum_{k=1}^{128} |x_{i,k} - \hat{x}_{i,k}| \quad (2)$$

$$\mathcal{L}_{\text{VAE}} = w_{\text{KL}} \cdot \mathcal{L}_{\text{latent}} + \mathcal{L}_{\text{reconstruction}} \quad (3)$$

$$\mathcal{L}_{\text{classification}}(y, \hat{y}) = -\log \hat{y}_y \quad (4)$$

$$\mathcal{L}_{\text{total}} = \mathbb{E}[\mathcal{L}_{\text{classification}} + w_{\text{VAE}} \cdot \mathcal{L}_{\text{VAE}}] \quad (5)$$

where

- $x \in \mathbb{R}^{6 \times 128}$: Input data
- $\hat{x} \in \mathbb{R}^{6 \times 128}$: Reconstruction of the input produced by the decoder
- $\mu = (\mu_1, \dots, \mu_d)$: Mean vector of the latent space distribution (output by encoder)
- $\log \sigma^2 = (\log \sigma_1^2, \dots, \log \sigma_d^2)$: Log variance vector of the latent space distribution
- d : Dimensionality of the latent space
- w_{KL} : Weighting factor applied to the latent (KL divergence) loss
- w_{VAE} : Weighting factor for the VAE loss in the total loss

- $y \in \{0, 1, \dots, 11\}$: True class label (sparse integer form)
- $\hat{y} \in [0, 1]^{12}$: Predicted class probabilities (softmax outputs), with $\hat{y}_c$ being the probability for class c
- 12: Number of classes for classification
- $\mathbb{E}[\cdot]$: Mean over the batch.

We will now investigate the effect of some hyperparameters on the performance of the DB-VAE. Starting with the choice of w_{VAE} , it seems clear that $w_{VAE} = 0$ would cause the model to train only for classification, similar to the CNNs in Section 2.3. While adaptive resampling would be active, we expect it to resemble a uniform distribution because feature extraction is initialized randomly and does not get trained. As $w_{VAE} \rightarrow \infty$, we expect the latent variables to represent features better and better and, therefore, the debiasing resampling mechanism to get more effective, likely at the cost of classification accuracy.

The following figure shows six experiments with $w_{VAE} \in \{0, 0.1, 1, 10, 100, 1000\}$, each of which comprises 5 independent iterations over 5 different random splits (as in Task 1).

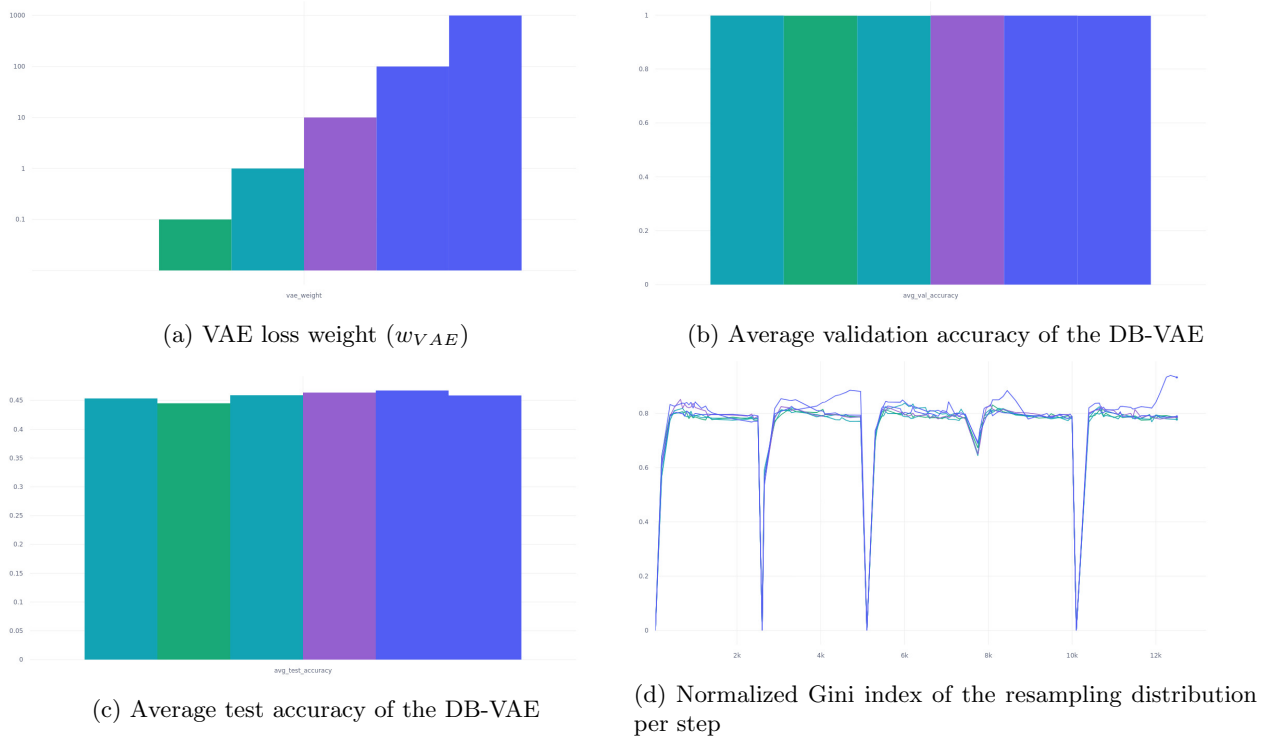


Figure 12: DV-VAE performance depending on w_{VAE}

Firstly, we notice that the average validation loss is 100% for all models. Even when $w_{VAE} = 1000$, the classification capability on the validation data is optimal. We might interpret this as some degree of independence between classification loss and VAE loss in the sense that reducing classification loss does not get penalized much with increased VAE loss. Otherwise, $w_{VAE} = 1000$ would cause large penalties on the total loss (cf. Eq. (5)), making an accuracy of 100% unlikely. Still further investigation is needed to support this claim.

Secondly, the average test accuracy is consistently around 45%, showing no significant improvement over the models from Task 1.

Lastly, we monitor the (normalized) Gini inequality index (cf. [6]) of the distribution used for resampling. This is a measure of inequality in a distribution (reaching a minimum of 0 for a uniform distribution and a maximum of 1 for a Dirac distribution). The five sections in the graph correspond to the five independent training cycles (for the five random splits). Hence, the spikes do not indicate a sudden jump during training, but merely resemble the fact that the resampling distribution gets initialized uniformly before the first epoch. The fact that the index stabilizes between 0.7 and 0.9, even when using $w_{VAE} = 0$, proves another

assumption incorrect: $w_{VAE} = 0$ does not necessarily cause the resampling distribution to be close to uniform. While feature extraction does get initialized randomly, it does not necessarily cause the latent variables to be distributed close to uniform. They might even degenerate during training in a random, non-uniform direction.

To actually compare the performance of a DB-Vae with debiasing resampling versus just uniform sampling of the training batches, we introduce a new hyperparameter $\alpha \in [0, 1]$, which controls the "aggressiveness" of debiasing in resampling. This is achieved by using the convex combination

$$p = \alpha p_{inverse} + (1 - \alpha) p_{uniform} \quad (6)$$

of the actual debiasing sampling distribution $p_{inverse}$ (calculated as described above) and a uniform distribution $p_{uniform}$ for drawing a random batch, essentially dampening the debiasing effect by the factor α . Note that $\alpha = 0$ completely disables the debiasing mechanism as batches are just drawn uniformly from the training data, whereas $\alpha = 1$ yields the original debiasing algorithm without dampening.

We now fix $w_{VAE} = 1000$ (as this did not decrease the classification capability earlier, and we want to make sure to learn relevant features). For $\alpha \in \{0, 0.2, 0.4, 0.6, 0.8, 1\}$ we run experiments (each time training five independent times over five random splits as before). The following figure shows the results.

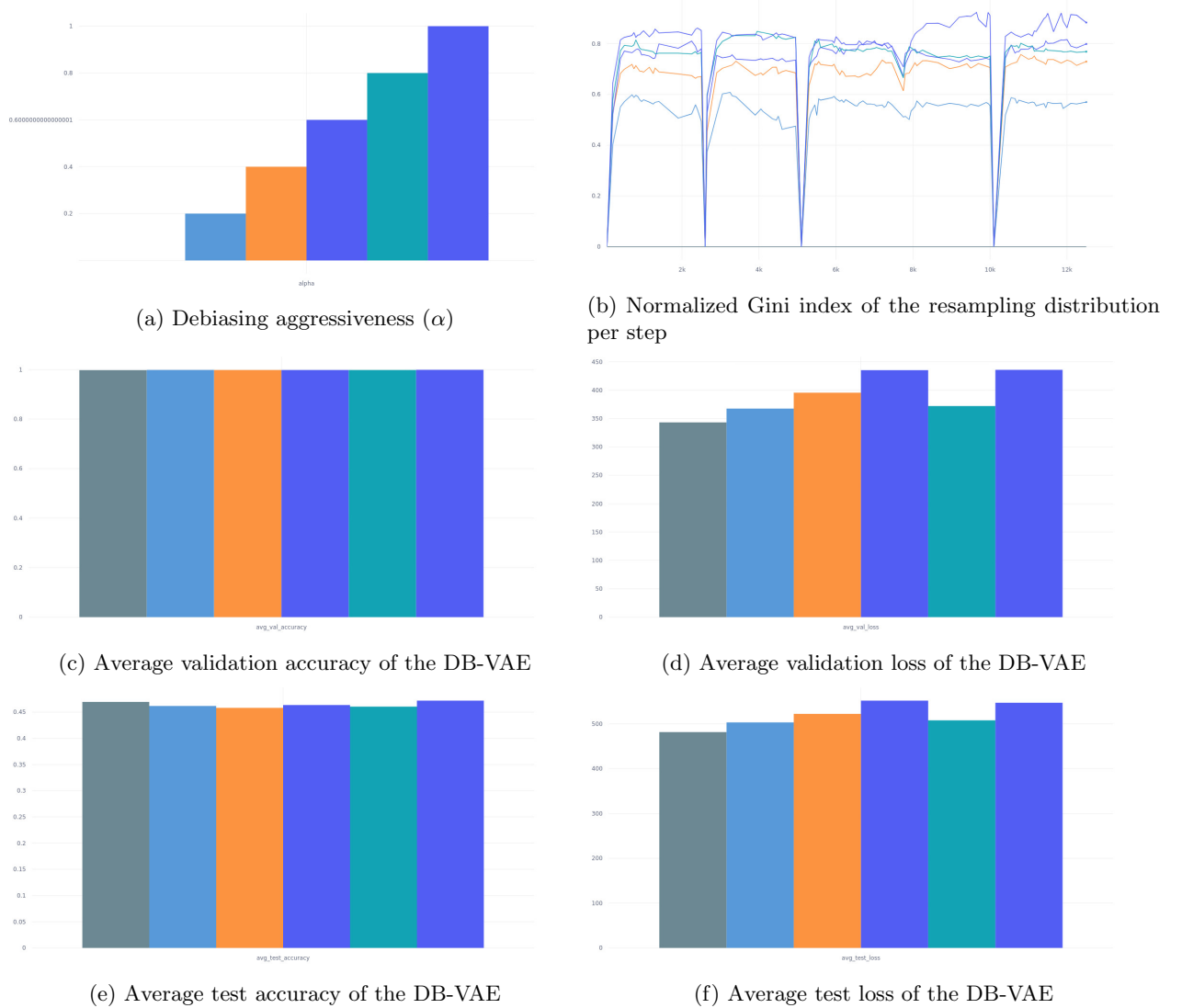


Figure 13: DV-VAE performance depending on α

We observe that the Gini index now depends very clearly on α , in particular it is zero when α is zero. This was to be expected in view of Eq. (6).

Apart from that, we do not see any significant impact of varying α on validation or test accuracy. This observation is not particularly expressive for the validation accuracy, because it is constant 100% in all models again, and a potential impact of varying α might just be too weak to counter the effective learning. The validation data might just be too similar to the training data in all five splits.

More interestingly, debiasing did not affect the test accuracy either. Hence, we suppose that the test data does not just resemble a redistribution of the training data (in which case debiasing would indeed be expected to be more effective), but has structural features distinct from all training data. The different ways these two data sets were created support this hypothesis, but deeper analysis into their "distinctiveness" may be needed.

Lastly, we are now able to compare the loss, as its definition is independent of α . We can observe a small increase as α increases from 0 to 0.6, both on the validation and the test data. With $\alpha = 0.8$ the loss decreases a little and with $\alpha = 1$ it is, again, similar to the loss at $\alpha = 0.6$. One possible explanation for the increase might be the fact that the same features that are overrepresented in the training data are also overrepresented in the validation and even the test data (e.g., class 0, i.e., all brakes intact). Thus, mitigating the overrepresentation during training via debiasing resampling might be penalized in the evaluation. This seems reasonable in view of Task 1, where the best models according to the test accuracy were just guessing label 0 each time. However, the increase in the loss is not very large, so further investigation on the significance as well as possible causes is necessary. (Martin)

4 Summary

When training neural networks of the type: FCN, CNN and LSTM on the braking data, all performed similarly with slight advantage for the LSTM. We experienced a strong bias towards the the outcome "class 0" or equivalently "all brakes work properly". The networks were trained using the random split validation method and have shown very good results with the validation data, but performed poorly on the test data, showing a missing ability to generalize.

Since the models were not able to generalize from the training data, the best performing neural networks learned the strategy to always guess the same label. The most straightforward explanation lies within the data structure, with the label taking up around 44% of the training and 59% of the test data.

The pursuit to tackle the bias with different techniques has not been successful. Preprocessing the data by the min-max-normalization and standardization actually hindered the neural networks and lead to a poorer performance across all versions. The outcomes did not allow for a deeper analysis of the driving force behind this decline due to very different results within network type and processing method without a clear trend. Data augmentation offered minimal advantages in test accuracy for FCN and LSTM, whilst the CNN had a disadvantage of the same magnitude. Depending on the model, there were different augmentations with the best fit. For the FCN flipping the data worked best, the CNN saw the least depreciation with amplitude scaling and the the LSTM profited of the zoom in. It was also the LSTM with the zoom in augmentation that has shown the overall best performance across all possible tests in this work (specifically it has the highest accuracy of all sets that did not guess zero every time). Also the augmentation helped fitting better to the test data than it helped with generalization, once again missing the major goal.

The autoencoder heavily resampled the data, as indicated by the Gini index. It still failed to help generalize, with the validation and test accuracy being almost unchanged. Further analysis actually supported the theory that test and training data have deeply embedded features that inhibit the models generalization, such as the big difference between them.

As a conclusion it is preferable to address the underlying data first before applying those techniques further. The work suggests, that the total lack of different labels in the test set as well as missing representation of car features and experiment conditions is a key problem that can not be solved by data manipulation techniques alone.

Obviously more evenly distributed data and filling the data gap with more experiments would be a direct solution. This would be associated with high cost and time. As a quicker and cheaper option based on the available data it would be interesting to resample test and training data. Alternatively splitting the data in different classes depending on vehicle weight etc. would enable a more specific accuracy analysis. Thereby it could be tested if the poor performance is mainly based on the dataset structure. (Ben)

5 References

References

- [1] accessed on 17 June 2025. 2020. URL: <https://stackoverflow.com/questions/62382224/why-the-lstm-autoencoder-use-relu-as-its-activation-function>.
- [2] accessed on 17 June 2025. 2024. URL: https://www.tensorflow.org/api_docs/python/tf/keras/activations.
- [3] accessed on 17 June 2025. 2025. URL: <https://www.comet.com/docs/v2/guides/optimizer/configure-optimizer/#the-algorithm-field>.
- [4] Alexander et al. Amini. “Uncovering and Mitigating Algorithmic Bias through Learned Latent Structure”. In: *Proceedings of the 2019 AAAI/ACM Conference on Artificial Intelligence, Ethics, and Society (AIES)* (2019).
- [5] Marina Dolokov. *Laboratory 2: Computer Vision — Part 2: Debiasing Facial Detection Systems*. accessed on 17 June 2025. 2025. URL: https://github.com/marinadolokov/2025-WIA-Deep-Learning/blob/main/Coding%5C%20Labs/lab3/solutions/TF_Part2_Debiasing_Solution.ipynb.
- [6] *Gini-Koeffizient*. accessed on 17 June 2025. 2025. URL: https://de.wikipedia.org/wiki/Gini-Koeffizient#Normierter_Gini-Koeffizient.
- [7] Coşku Can Horuz et al. *The Resurrection of the ReLU*. 2025. arXiv: 2505.22074 [cs.LG]. URL: <https://arxiv.org/abs/2505.22074>.